

Jenkins

Lab 5

Create a Pipeline that executes tests described on the next page when triggered manually from Jenkins UI.

The pipeline must have at least the following stages:

1. Build
2. Program Pico using debugger
3. RobotFramework tests

You need to show that your pipeline works to the instructor.

For general information about Jenkins pipelines see:

<https://www.jenkins.io/doc/book/pipeline/>

Software on Pico responds to simple AT-commands. In this exercise the only commands needed are AT and AT+SEND. When Pico receives characters, it echoes (=sends) them back. This feature is called local echo, which makes it easier to test the software using a serial terminal program.

AT+SEND sends text using Morse code on the onboard LED. In Morse only letters from A-Z, numbers 0-9, and a space are available. All white space (space, tabs, etc.) is converted to spaces, letters from A-Z are converted to upper case, and all other characters are converted to 'X' and. The response to the command starts with SENT and follows with the converted text and OK on separate lines. The response always contains both carriage return and line feed characters. Command to the device can end with either or both line end characters.

AT+SEND="hello, world!"

Command (from your computer)

AT+SEND="hello, world!"

Echo – Pico echoes the sent/typed characters back

SENT="HELLOX WOLRDX"

Response (two lines)

OK

The test case file communicates with Raspberry Pi Pico over USB serial port. To verify that connection to the device has been established AT with no parameters must be used. The response to AT is OK if connection is established. Device responds to commands that don't start with AT with ERROR. Device responds with INVALID if the command starts with AT but is not recognized by the device.

The program has following properties:

- Local echo is on by default.
 - Local echo (on) means that received characters are immediately sent back after they have been received. Any response the software may produce follows immediately after the echoed characters when a line end is received. Local echo makes it easier to work with terminal program that don't have a separate transmit and receive windows.
 - When local echo is off received characters are not echoed back.
- ATE command. ATE can take three forms:
 - ATE responds with the current state of local echo (ON/OFF) followed by OK.
 - ATE0 switches local echo off. Responds with OK.
 - ATE1 switches local echo on. Responds with OK.
- ATW command. ATW can take two forms:
 - ATW responds with the current dot length in ms followed by OK.
 - ATW<length in ms> sets a new dot length in ms. Allowed values are 10 – 1000. If value is valid response is OK. If value is outside the allowed range the response is INVALID. For example: ATW50 sets dot length to 50 ms.

The default value after reboot is 100 ms.

What to test:

- Test with texts of three different lengths:
 - short – less than 10 characters
 - medium – less than 20 characters
 - long – about 30 characters
- Test with three different speeds
 - Fast – 20 ms
 - Medium – 100 ms
 - Slow – 500 ms
- Tests must contain all combinations of the above
- Slow tests must be tagged

Pico sends the response is sent after it has finished transmitting morse code with the led. This means that the response time depends on the length of the text to transmit and the transmit speed. A good test adapts to the expected timeout. A fast test should have shorter timeout than a longer one. Pyserial allows you to change the read timeout: https://pyserial.readthedocs.io/en/latest/pyserial_api.html#serial.Serial.timeout

Setup

In this exercise we run Jenkins and Gogs (a Git server) in containers. On the first run we need to configure both Jenkins and Gogs. After the initial setup the container can be stopped and started at will. The settings are retained in named volumes so there is no need to reconfigure the services.

To keep the URLs consistent both in containers and on the host computer you need to edit `/etc/hosts`. On linux and OSX the file path is `/etc/hosts` (what a surprise!). On windows the file path is:

`C:\Windows\System32\drivers\etc\hosts`. Note that you need administrator privileges to edit the file. Add the following lines to `/etc/hosts`:

```
# Reroute jenkins and gogs to localhost
127.0.0.1 jenkins
127.0.0.1 gogs
```

Windows version of Docker installs Docker compose automatically. If you use Linux, you need to install Docker compose manually after installing Docker.

Copy Dockerfile and docker-compose.yml to the same directory. Open command prompt/terminal, go to the directory and execute:

```
docker compose up
```

Now your containers are up and running and you can perform the initial configuration of Jenkins and Gogs. Following chapters explain how to setup the services.

When you want to stop your containers just execute:

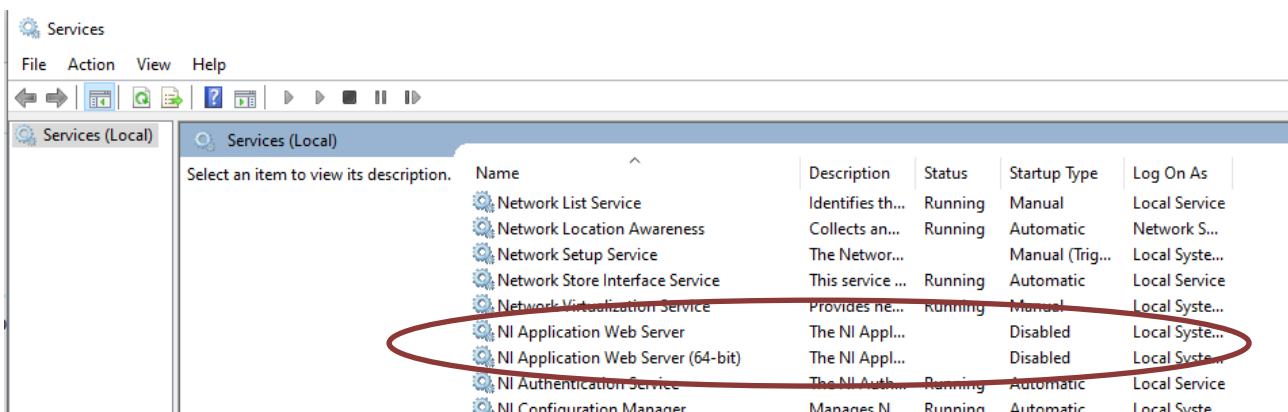
```
docker compose down
```

You can start the contained in detached mode by executing:

```
docker compose up --detached
```

In detached mode you won't see the console messages. You don't really need the console messages after the initial setup so once your containers are setup start the containers in detached mode.

If you have Labview installed you must stop, or stop and disable **NI Application Web Server**. By default NI installs web server that uses port 8080 which will prevent Jenkins from running in the default port.

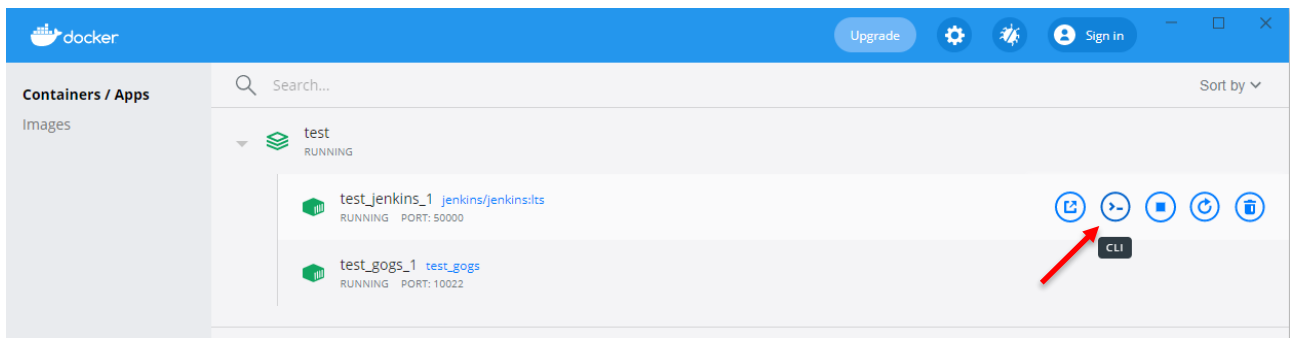


Navigate to `jenkins:8080` in your browser.

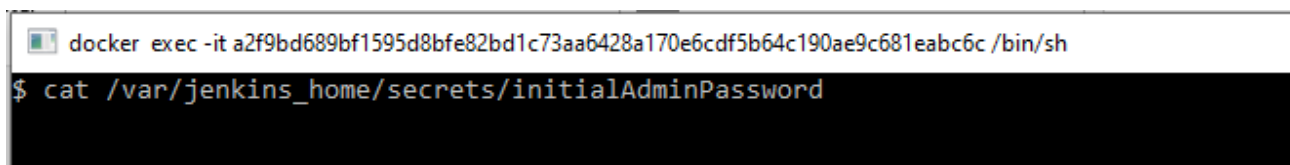
```
Windows PowerShell
jenkins_1 | 2021-04-20 06:26:45.473+0000 [id=31] INFO jenkins.InitReactorRunner$1#onAttained: System config lo
ad
jenkins_1 | 2021-04-20 06:26:45.474+0000 [id=43] INFO jenkins.InitReactorRunner$1#onAttained: System config ad
apted
jenkins_1 | 2021-04-20 06:26:45.475+0000 [id=31] INFO jenkins.InitReactorRunner$1#onAttained: Loaded all jobs
jenkins_1 | 2021-04-20 06:26:45.475+0000 [id=33] INFO jenkins.InitReactorRunner$1#onAttained: Configuration fo
r all jobs updated
jenkins_1 | 2021-04-20 06:26:45.485+0000 [id=56] INFO hudson.model.AsyncPeriodicWork#lambda$doRun$0: Started D
ownload metadata
jenkins_1 | 2021-04-20 06:26:45.494+0000 [id=56] INFO hudson.util.Retrier$start: Attempt #1 to do the action c
heck updates server
jenkins_1 | 2021-04-20 06:26:45.949+0000 [id=38] INFO jenkins.install.SetupWizard#init:
jenkins_1
jenkins_1
jenkins_1
jenkins_1
jenkins_1 | Jenkins initial setup is required. An admin user has been created and a password generated.
jenkins_1 | Please use the following password to proceed to installation:
jenkins_1 | d358bcd5f4dc44f89112b785211dd56c
jenkins_1
jenkins_1 | This may also be found at: /var/jenkins-home/secrets/initialAdminPassword
jenkins_1
jenkins_1
jenkins_1
jenkins_1 | 2021-04-20 06:26:54.277+0000 [id=43] INFO jenkins.InitReactorRunner$1#onAttained: Completed initia
lization
jenkins_1 | 2021-04-20 06:26:54.296+0000 [id=221] INFO hudson.WebAppMain$3#run: Jenkins is fully up and running
```

Note that the browser will complain about insecure connection. Just ignore the warning because we are actually connecting to localhost.

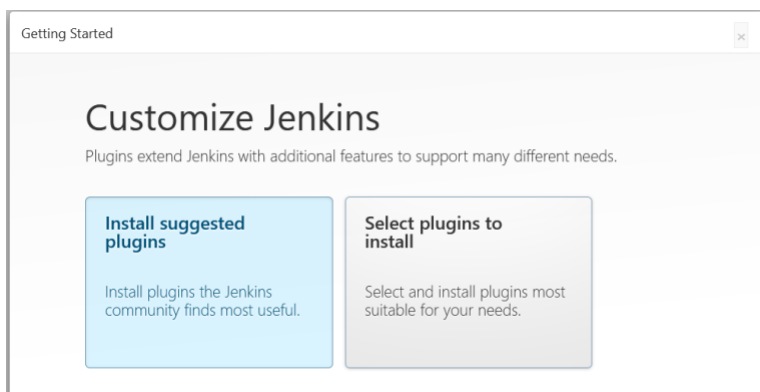
If you can't find the initial admin password from the logs you can run a console in the container. See below.



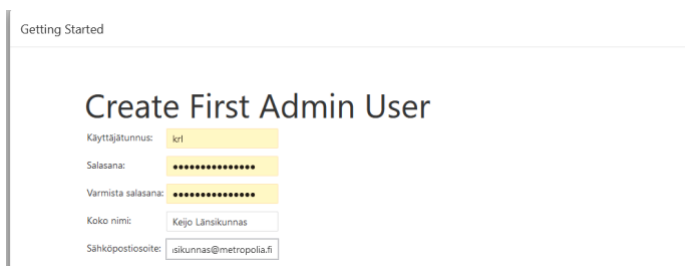
Type command in the console. Note that there is no tab-completion or command history in the Jenkins container's shell.



After entering the initial password, you get the following screen. Select install suggested plugins.



After the plugins have been installed you need to create the first admin user. It is important to keep this password safe because the initial password is a onetime password that will not work after this stage.



After creating Admin user, you need to configure the URL used to access Jenkins. Accept the default URL.

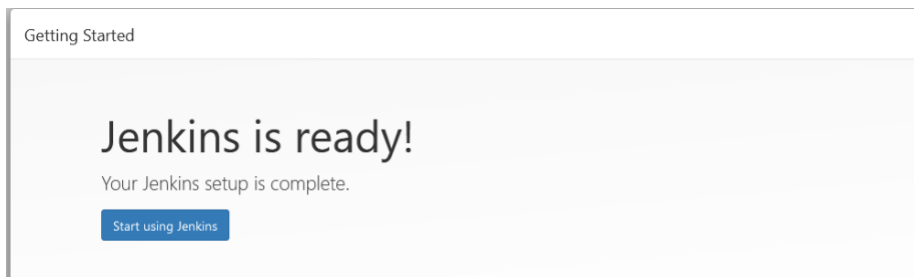
Getting Started

Instance Configuration

Jenkins URL:

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the `BUILD_URL` environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.



At this point Jenkins is NOT ready. Click **Start using Jenkins** and install some additional plugins according to the instructions below. Depending on your browser settings you may see part of Jenkins UI in English and part in some other language.

 **Jenkins**

Dashboard ▾

 Uusi Item

 Käyttäjät

 Käännöshistoria

 Hallitse Jenkinsia

 My Views

 Lockable Resources

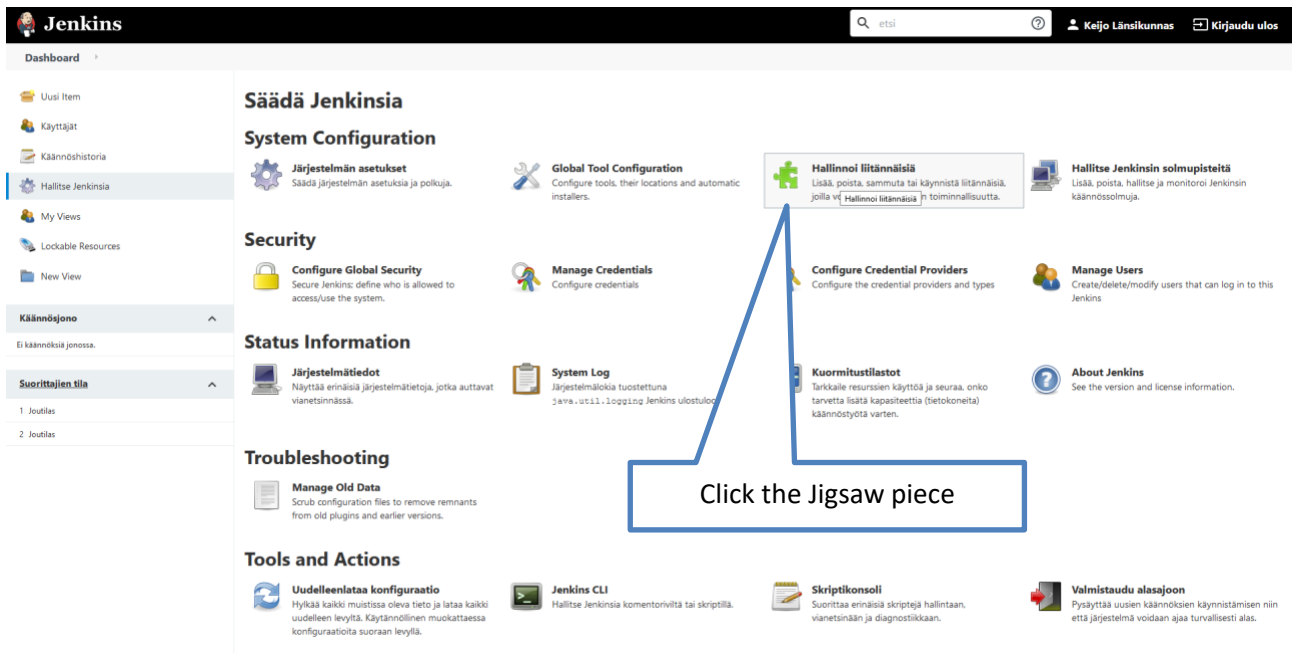
 New View

Click the gear icon

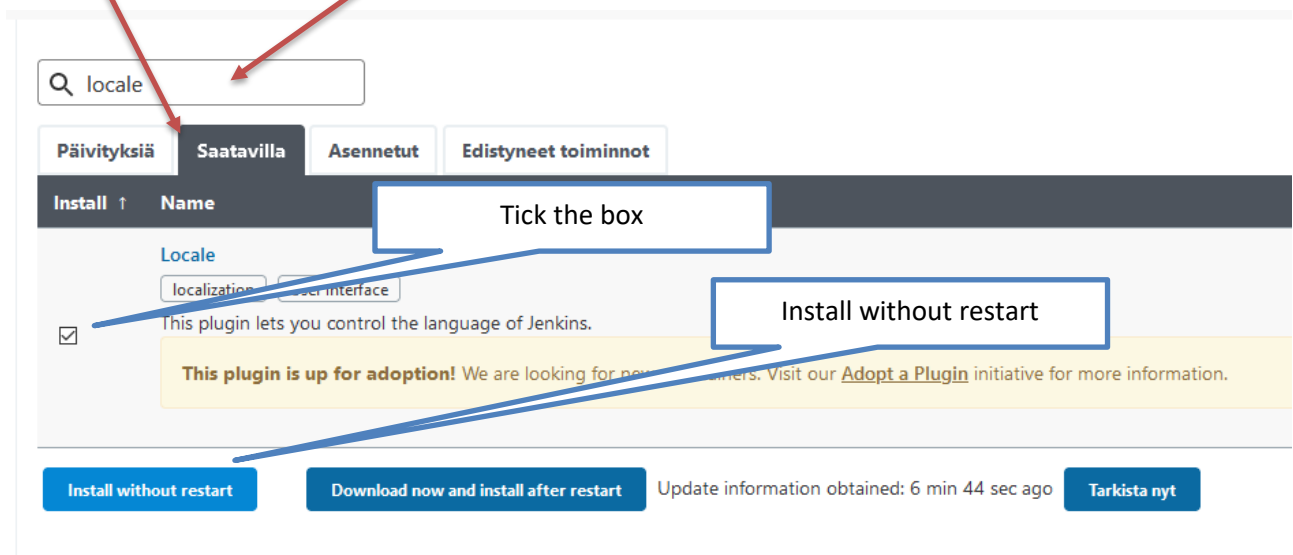
What a mess!

Käännösjono ^
Ei käännöksiä jonossa.

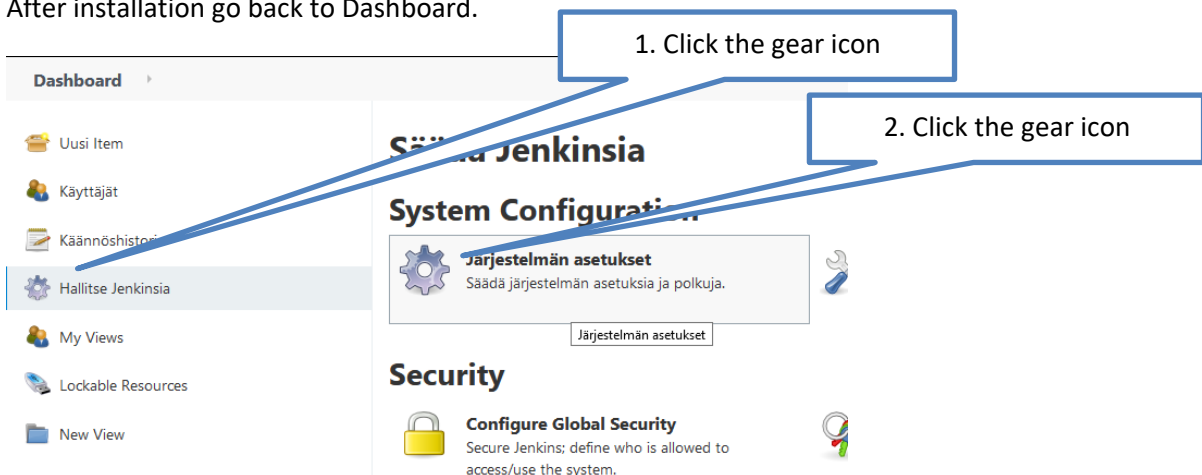
Suorittajien tila ^
1 Joutilas
2 Joutilas



Select second tab and type locale in the search box.



After installation go back to Dashboard.



Scroll down to **Locale** and do the following changes:

—

Locale

Default Language

en

☒ Ignore browser preference and force this language to all users

Click **Apply** at the bottom of the page.

Now your Jenkins UI is consistently in English.

 **Jenkins**

Search (🔍+K) ?

Keijo Lämsikunnas ▾ log out

Dashboard > Manage Jenkins

+ New Item

👤 People

📅 Build History

🕒 Project Relationship

🔍 Check File Fingerprint

⚙️ Manage Jenkins

📄 My Views

Build Queue ^

Build Executor Status

📄 Built-In Node

1 Idle

2 Idle

📄 my_agent (offline)

Manage Jenkins

Search settings /

Building on the built-in node can be a security issue. You should set the number of executors on the built-in node to 0. See [the documentation](#).

Manage Dismiss

System Configuration

 **System**
Configure global settings and paths.

 **Tools**
Configure tools, their locations and automatic installers.

 **Plugins**
Add, remove, disable or enable plugins that can extend the functionality of Jenkins.

 **Nodes**
Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

 **Clouds**
Add, remove, and configure cloud instances to provision agents on-demand.

 **Appearance**
Configure the look and feel of Jenkins.

Security

 **Security**
Secure Jenkins; define who is allowed to access/use the system.

 **Credentials**
Configure credentials

 **Credential Providers**
Configure the credential providers and types

 **Users**
Create/delete/modify users that can log in to this Jenkins.

 **In-process Script Approval**
Allows a Jenkins administrator to review proposed scripts (written e.g. in Groovy) which run inside the Jenkins process and so could bypass security restrictions. 1 scripts pending approval.

Jenkins needs a repository to connect to so the next step is to configure Gogs. Gogs is a light-weight Git server that is easy to manage.

Gogs setup

Initial setup is done on a single page. Settings should be as shown below. These settings can't be changed from GUI later! Open a new tab in browser and go to <http://gogs:3000>

The screenshot shows the 'Install Steps For First-time Run' page for Gogs. It contains several sections of settings with blue callout boxes providing instructions:

- Database Settings:**
 - Database Type:** SQLite3 (Selected). Annotation: "Select SQLite3. Don't change the path".
 - Path:** data/gogs.db
- Application General Settings:**
 - Application Name:** Gogs
 - Repository Root Path:** /data/git/gogs-repositories
 - Run User:** git
 - Domain:** gogs. Annotation: "Set Domain to gogs".
 - SSH Port:** 10022. Annotation: "Set SSH Port to 10022".
 - ☐ Use Builtin SSH Server
- HTTP Port:** 3000
- Application URL:** http://gogs:3000/. Annotation: "Set URL to: http://gogs:3000/"
- Log Path:** /app/gogs/log
- ☐ Enable Console Mode
- Default Branch:** main. Annotation: "Set default branch to: main"
- Optional Settings:**
 - Server and Other Services Settings
 - ☐ Enable Offline Mode
 - ☐ Disable Gravatar Service
 - ☐ Enable Federated Avatars Lookup
 - ☐ Disable Self-registration
 - ☐ Enable Captcha. Annotation: "... untick Captcha"
 - ☐ Enable Require Sign In to View Pages
 - Admin Account Settings

At the bottom is a blue button labeled "Install Gogs".

After installation you will be taken to sign in. At this point there is no user or admin account, so you need to sign up first.

[Home](#)[Explore](#)[Help](#)[Register](#)[Sign In](#)

Sign In

Welcome! We're glad that you chose Gogs, have fun and take care.

Username or email *

Password *

☐ Remember Me

[Sign In](#) [Forgot password?](#)

[Need an account? Sign up now.](#)

The first user account that you create will automatically get admin privileges.

Sign Up

Username *

Email *

Password *

Re-Type *

[Create New Account](#)

[Already have an account? Sign in now!](#)

When the account is created you can log in and start using Gogs.

Sign In

Username or email *

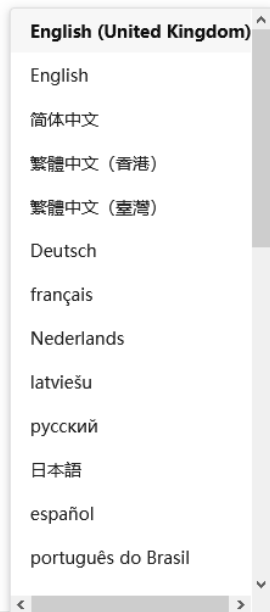
Password *

☐ Remember Me

[Sign In](#) [Forgot password?](#)

[Need an account? Sign up now.](#)

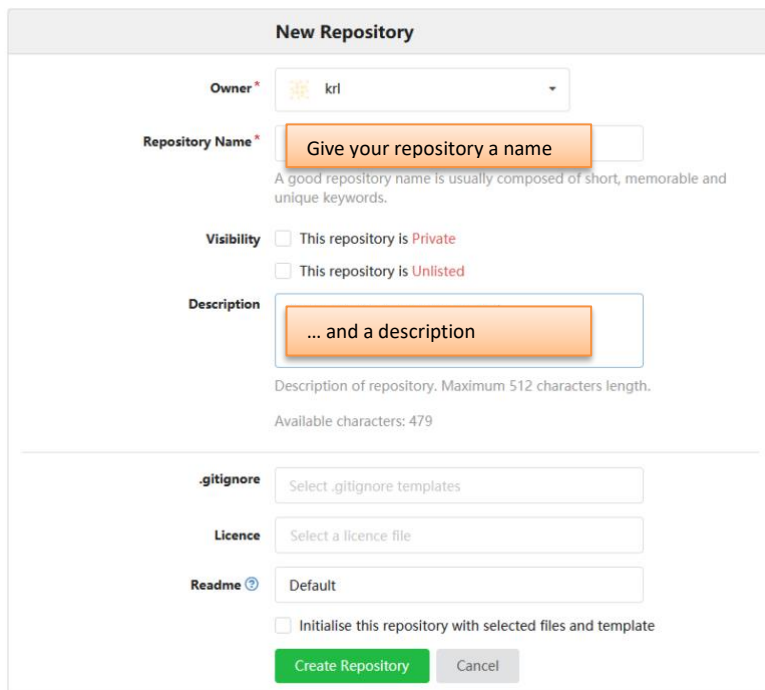
If your UI is not in the language you prefer you can change the language from the menu at the bottom of the page.



Page: 31ms Template: 2ms |  English (United Kingdom) | [Website](#)

Create a new repository

The following shows an example how to create a repository in Gogs and how to push an existing repository to Gogs. When you create a new repository set the name and description of the repository and leave the rest at their default values. This will create an empty repository.

A screenshot of the 'New Repository' form in Gogs. The form is titled 'New Repository' and contains several fields and options. The 'Owner' field is set to 'krl'. The 'Repository Name' field has a placeholder 'Give your repository a name' and a note: 'A good repository name is usually composed of short, memorable and unique keywords.' The 'Visibility' section has two checkboxes: 'This repository is Private' (unchecked) and 'This repository is Unlisted' (unchecked). The 'Description' field has a placeholder '... and a description' and a note: 'Description of repository. Maximum 512 characters length. Available characters: 479'. Below the description field are three more fields: '.gitignore' (with a dropdown 'Select .gitignore templates'), 'Licence' (with a dropdown 'Select a licence file'), and 'Readme' (with a dropdown 'Default'). At the bottom, there is an unchecked checkbox 'Initialise this repository with selected files and template' and two buttons: 'Create Repository' (green) and 'Cancel' (grey).

After creating the repository, you get the quick guide of how to populate the repo with data.

Quick Guide

Clone this repository Need help cloning? Visit [Help!](#)

[HTTP](#) [SSH](#) `http://gogs:3000/krl/morse.git`

Create a new repository on the command line

```
touch README.md
git init
git add README.md
git commit -m "first commit"
git remote add origin http://gogs:3000/krl/morse.git
git push -u origin master
```

Push an existing repository from the command line

```
git remote add origin http://gogs:3000/krl/morse.git
git push -u origin master
```

This is just an example! Check the commands from your own gogs. The commands depend on the name of your repository!

Extract the files from zip file in the workspace and create a new repository of the files in the zip.

Setup credentials

To make ssh access easier we make Jenkins accept first connection from a previously unknown host. Go to Manage Jenkins→Security and set Git host key verification to accept first connection.

Git Host Key Verification Configuration

Host Key Verification Strategy ?

Accept first connection

Save

Apply

Credentials are needed to allow Jenkins to access repository(ies). Following shows how to add ssh-keypair to Jenkins and Gogs. The keypair is generated in Jenkins container. Open terminal in Jenkins container and run the commands highlighted in red. Leave terminal open, you'll need the output in the setup.

jenkins-1
ff3173a89731
jenkins/jenkins:its
Running
0.19%
50000:50000
Show all ports (2)
3 hours ago

- View details
- View image packages and CVEs
- Copy docker run
- Open in terminal
- View files
- Pause

Mem 3.40 GB CPU 0.00% Disk 55.72 GB avail. of 62.67 GB Not signed in

```
$ ssh-keygen
```

Generating public/private rsa key pair.

Enter file in which to save the key (/var/jenkins_home/.ssh/id_rsa):

Created directory '/var/jenkins_home/.ssh'.

Enter passphrase (empty for no passphrase):

Enter same passphrase again:

Your identification has been saved in /var/jenkins_home/.ssh/id_rsa

Your public key has been saved in /var/jenkins_home/.ssh/id_rsa.pub

The key fingerprint is:

...

```
$ cd /var/jenkins_home/.ssh
```

```
$ cat id_rsa
```

```
-----BEGIN OPENSSH PRIVATE KEY-----
```

```
b3B1bnNzaC1rZXktdjEAAAAACmFlczI1Ni1jdHIAAAAGYmNyeXB0AAAAGAAAAABAdPpNydc
```

```
<text removed...>
```

```
G/f/YMg+s5suodqKNahwnVTcFb46bY170C6lcUWurujLEoiB/SZ4V3Iao1v/Egtn5sj8XB
```

```
00IRsPckXD6Xx+AoQiMgzUPt3QZhAcB0trHwZSk1CMAQsrD6eHiVWx/fhpAUDFo+vKVeB4
```

```
6pQ9JWI12yRXJSxfcdTx0R0JukRyk6uyjYZMrC1gf3f0LbYdnW6fGAZnDztWE5A15AviKv
```

```
NLIRhMBSMDFEAt1T7TwZwsb3+8g=
```

```
-----END OPENSSH PRIVATE KEY-----
```

```
$ cat id_rsa.pub
```

```
ssh-rsa
```

```
AAAAB3NzaC1yc2EAAAADAQABAAQGDPPQ29j7MoiIvsnc0P/ae1LJ+lw7fYxeAWznRRKRw2eC/aZDI
```

```
uU0tafs+EA+YA55ND0rv80kvrG1wK4HnXhqR6acczsvbuQICJy9qHnBU5KbvVJuNLtjZRcxzo4Fu1c
```

```
T2Zf1+X5IaoKvbXRmLMRgwhcW/dR502vNKofSsRyIXUfgozN1N1Q8f3i1EX2k9qUAekTOV1ALKdL/Gv
```

```
Nu1Utxwuq2DaVFvVe3FioUpaV8cuI25TA1x+1L1SWToqr7R1eIVhT6N7RY1lBqL6sdMHx3CJy+J1CjL
```

```
8AZKrL3b0WZ31ydRJbK8KaI0fvJyTHk1ZEgAaSzphfZgFe+HpNkLhNBgGpSunEeqgnmpZ0tdrI+Lool
```

```
Ij+j3Uv8xI0YdXanDZUaHq6y4vTrdep/K0cyPFJEFT1yB93YTLz63Vg66JVYVGKKRba3Q13e0gQgaP
```

```
+4Wz44nJioB8MT4MkTjT1F/K1KJvk9ITyjQ0TLXr//jQWW5CWGHhALpyuGMXW/ezYjLcc=
```

```
jenkins@jenkins
```

```
$
```

Private key of the pair is added to Jenkins credential provider and public key is added to Gogs.



Credentials

T	P	Store ↓	Domain	ID
		System	(global)	236d5fc'

Add credentials

Stores scoped to Jenkins

New credentials

1. Kind
SSH Username with private key

Scope ?
Global (Jenkins, nodes, items, all child items, etc)

ID ?

Description ?

2. Username
git

☐ Treat username as secret ?

Private Key

☒ Enter directly

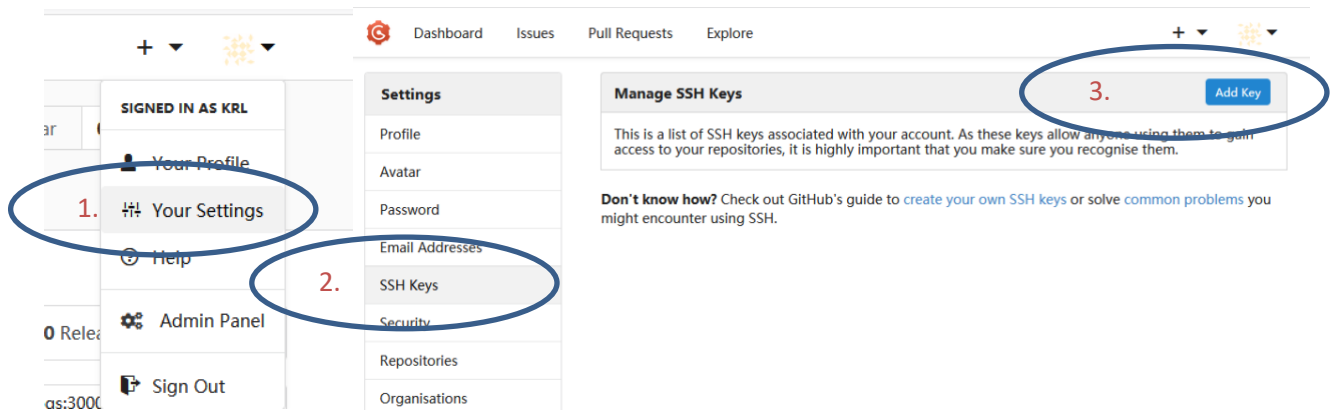
Key
No Stored Value

4. Paste the private key to the input box that opens. Private key must include the BEGIN and END lines of the private key.

3.

Passphrase

Copy the public key from Jenkins terminal to the clipboard and switch to Gogs. Go to Your settings and select SSH Keys and click Add Key.



Note: Adding the key in user settings gives Jenkins access to all your repos. It also allows write access which is needed with Blue Ocean pipeline editor (which we don't use...).

Name the key and paste the key you copied from Jenkins container terminal to Content box.

Manage SSH Keys

Add Key

This is a list of SSH keys associated with your account. As these keys allow anyone using them to gain access to your repositories, it is highly important that you make sure you recognise them.

Don't know how? Check out GitHub's guide to [create your own SSH keys](#) or solve [common problems](#) you might encounter using SSH.

Add SSH Key

Key Name

Jenkins Blue Ocean access

Content

```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCFh0Ms/y+aVjm5SlghRnIHcHZ
/1O4ThUDexm5i7LxZSne7jmfYlnrI3Lzz7lq5p0EPaVJm2Dpr2Pelw8CxN4Vy8n6yQ7
/K6+OTol5UmXBblnbAVnBLO54A1YyaezXdbo7thi+sPACy+2+/gqjldt
/DDMm5rSVJmmk+bjWbsBBYf+2fbJFagLMrVWuez2aHjqGNov5pLp
/7uGiNZwlqY3PjpHDVXOz9AT6eFs2D6D3AmMRHq1cwsI0zjOzVsvrOPZDJAnvZCOhIC31tsLpYNUmyK0
/099Av/y4C+46jzQULTO4sC9nxORYKs+oDNXyAibM2IXXi7cMbwoAVQQAQVz.krl@jenkins:8080
```

Add Key

Then click Add Key.

New SSH key 'Jenkins Blue Ocean access' has been added successfully!

Manage SSH Keys

Add Key

This is a list of SSH keys associated with your account. As these keys allow anyone using them to gain access to your repositories, it is highly important that you make sure you recognise them.

Jenkins Blue Ocean access

SHA256:17mH3OhuXkq7DttGi1NtHTFm8OGxmAMJC1F9HPhrU

Added on Apr 20, 2021 — ⓘ No recent activity

Delete

Don't know how? Check out GitHub's guide to [create your own SSH keys](#) or solve [common problems](#) you might encounter using SSH.

Now you are all set with installing the access credentials and you are ready to create a pipeline.

Create a Pipeline

Go to dashboard and click “New Item”. Name your item and select **Pipeline** as the item type.

The screenshot shows the 'Create a Pipeline' form. On the left is a sidebar with 'Dashboard' and a list of options: '+ New Item', 'People', 'Build History', and 'Project Relationship'. The main area is titled 'Enter an item name' and contains a text input field with 'pico-robot-test' and a 'Required field' note. Below this are three selectable options: 'Freestyle project' (described as a classic, general-purpose job type), 'Pipeline' (described as orchestrating long-running activities), and 'Multi-configuration project' (described as suitable for projects with many configurations).

Press ok and then go the bottom half of the next view to setup pipeline type.

Pipeline

The screenshot shows the 'Pipeline' configuration form with several fields and annotations:

- Definition:** A dropdown menu set to 'Pipeline script from SCM'.
- SCM:** A dropdown menu set to 'Git'.
- Repository URL:** A text input field containing 'ssh://git@gogs:10022/krl/atcmd.git'. An annotation points to this field: 'You get the link from your repo in Gogs. The name shown here is an example. Your repository name may be different'.
- Credentials:** A dropdown menu set to 'git'. An annotation points to this field: 'Select the credentials you added earlier.'
- Advanced:** A section containing an 'Add Repository' button and a 'Branches to build' section.
- Branches to build:** A section with a 'Branch Specifier (blank for 'only')' dropdown menu set to 'main'. An annotation points to this field: 'Select branch to build. Usually 'main' or 'master''.

The screenshot shows the Jenkins Pipeline Configuration interface. At the top, there is a 'Repository browser' dropdown menu set to '(Auto)'. Below it is the 'Additional Behaviours' section with an 'Add' button. The 'Script Path' field is highlighted with an orange oval and contains the text 'Jenkinsfile'. An orange callout box points to this field with the text: 'Relative path in the repo to the pipeline definition file. Can have any name but it is typically called Jenkinsfile.' Below the 'Script Path' field is a checkbox labeled 'Lightweight checkout' which is checked. At the bottom, there are two buttons: 'Save' and 'Apply', both of which are circled in orange. A link labeled 'Pipeline Syntax' is also visible.

Your pipeline is ready to run. There is a simple Jenkinsfile in the attached zip file. If you added all files to the repo, you should now have a pipeline that is ready to run.

Test that your pipeline runs with the example Jenkinsfile. If your pipeline runs then change the pipeline to have stages: build, program and test.

Setup a node

First you need to setup a node for building the project. A node in Jenkins is a computer running an agent, a service that connects to Jenkins to receive commands to execute. A node is setup or configured in Dashboard→Manage Jenkins→Nodes

The screenshot shows the 'New node' configuration page in Jenkins. At the top, there is a breadcrumb trail: 'd > Manage Jenkins > Nodes > New node'. Below this is the 'New node' heading. The 'Node name' field is filled with 'test_agent'. The 'Type' section has a radio button selected for 'Permanent Agent'. Below this, there is a description: 'Adds a plain, permanent agent to Jenkins. This i' followed by a partially visible line 'of interaction with these agents, such as dynam'.

Name ?
test_agent

Description ?

Plain text [Preview](#)

Number of executors ?
1

Remote root directory ?
/Users/lansk/Documents/jenkins

Labels ?
pico-build

Usage ?
Only build jobs with label expressions matching this node

Launch method ?
Launch agent by connecting it to the controller

Availability ?
Keep this agent online as much as possible

Node Properties

☐ Disable deferred wipeout on this node ?
☐ Disk Space Monitoring Thresholds
☐ Environment variables
☐ Tool Locations

[Save](#)

Agent uses this path for the files it needs for build. For example, sources that are checked out.

Label is used to select node that is used for running the pipeline or a stage.

Agent connects to the controller (Jenkins). Jenkins considers agent to be offline if there is no connection and will not try to reconnect to the agent.

If agent is offline Jenkins will display instructions how to start agent on different platforms. Note that you need to have Java installed to run the agent. Java version 11 or newer is required to run the agent.

Run from agent command line: (Unix)

```
curl -sO http://jenkins:8080/jnlpJars/agent.jar
java -jar agent.jar -url http://jenkins:8080/ -secret d2ef8dae9a788e4140459de6f9b1dc5fe347e4454f0714b0f7ef216eab453ea0 -name "my_agent"
-workDir "/Users/lansk/Documents/jenkins"
```

Run from agent command line: (Windows)

```
curl.exe -sO http://jenkins:8080/jnlpJars/agent.jar
java -jar agent.jar -url http://jenkins:8080/ -secret d2ef8dae9a788e4140459de6f9b1dc5fe347e4454f0714b0f7ef216eab453ea0 -name "my_agent"
-workDir "/Users/lansk/Documents/jenkins"
```

Windows does not have curl installed by default. The easiest way to download agent.jar in Windows is to copy the address in browser and save agent.jar. The jar-file does not change so if you keep it safe you only need the second line to start the agent and you can make a batch-file of the second line to speed up stating the agent.

When agent is online you can see the status and the agent information in Dashboard→Nodes.

The screenshot shows the Jenkins Dashboard. On the left, the 'Nodes' tab is selected, showing a list of nodes: 'Built-In Node' and 'my_agent'. The 'my_agent' node is circled in orange. Below this, the 'Build Executor Status' section shows the status of the 'my_agent' node, which is 'Idle'. On the right, the 'Nodes' table is displayed, showing the details of the 'my_agent' node, which is circled in orange. The table has columns: 'S', 'Name', 'Architecture', and 'Cloc Difference'. The 'my_agent' node is listed with 'Mac OS X (aarch64)' architecture and 'In' status. Below the table, the 'Data obtained' section shows '14 min' and '14'.

S	Name ↓	Architecture	Cloc Difference
🖥️	Built-In Node	Linux (aarch64)	In
🖥️	my_agent	Mac OS X (aarch64)	In

Data obtained: 14 min, 14

Agent is offline:

The screenshot shows the Jenkins Dashboard with the 'my_agent' node listed as 'offline'.

We want all the steps of the pipeline to run our pico-build node so the first thing to change is the type of node that the pipeline can run on. The example pipeline can be run on any node:

```
1 pipeline {
2   agent any
3   stages {
```

In the pipeline display you find Pipeline Syntax which contains generators that can be used to create code for steps and directives controlling the pipeline execution. In the generator you choose what directive or step you want to use, fill in the information and press generate to output a snippet to copy to your Jenkinsfile.

The screenshot shows the Jenkins Pipeline Syntax page. The left sidebar contains a list of options: 'Status', 'Changes', 'Build Now', 'Configure', 'Delete Pipeline', 'Full Stage View', 'Rename', and 'Pipeline Syntax'. The 'Pipeline Syntax' option is selected and highlighted with an orange arrow. The main content area shows the 'Pipeline Syntax' page with a 'Back' button and a list of generators: 'Snippet Generator', 'Declarative Directive Generator', 'Declarative Online Documentation', and 'Steps Reference'. The 'Snippet Generator' option is highlighted with an orange arrow.

Set the pipeline to run only on a **pico-build** node.

Build step

The first stage is “build”. The build takes place in a docker container that is defined by Dockerfile and started using docker compose file. You can find both files in pico-build-docker.zip. Build can also be run manually:

- copy the two files to your source directory that contains Cmakelists.txt
- start terminal in the directory
- run: docker compose up
 - the compose file will start the container, mount the source directory to the container and run the build commands.
 - The build artefacts will go into cmake-build-debug directory that gets created during the build.

To automate build:

- Add the two files from pico-build-docker.zip to the top level of your repository.
- Make the first stage execute “docker compose up” either with **sh** or **bat** script depending on your platform.
- Test your pipeline. If build was successful, you should find checked out repo and the build artefacts in your agents working directory (remote root directory you chose when configuring the agent) after the pipeline finishes.

Program step

Connect the tricolor (yellow, black, orange) cable with female headers to D-port of Pico probe and the other end of the cable as shown in the image below.



To program the pico using debugger you need to install openocd 0.12 or later. On Linux and OSX use your package manager to install openocd. You can also find prebuilt binaries for Windows, OSX, and Linux here:

<https://github.com/xpack-dev-tools/openocd-xpack/releases/>

The prebuilt binaries are portable, meaning that there is no installer, and you can just extract them to the location of your choice. Openocd installations come with processor specific configuration files. Raspberry Pi Pico configuration file must be edited for programming to work correctly:

- Locate pico-debug.cfg
 - It is in directory scripts/board, which is typically under your openocd installation directory
- Change the last line to:
source [find target/rp2040.cfg]

By default, the commands are executed in the top-level of the checked-out repository. After installing openocd and editing pico-debug.cfg, test programming manually. Go to top-level of a successfully built project. Run:

Path to your openocd.exe. Can be omitted if openocd is in PATH

Script directory of your openocd

```
c:\bin\openocd-0.12.0-3\bin\openocd.exe -s c:\bin\openocd-0.12.0-3\openocd\scripts -f board/pico-debug.cfg -c "tcl_port disabled" -c "gdb_port disabled" -c "tcl_port disabled" -c "program \"cmake-build-debug/atcmd.elf\" \" -c reset -c shutdown
```

Attention! Path to executable to program uses unix-style path separator also on Windows

When programming works use snippet generator to turn the command line to Jenkins command (sh/bat) and add the command to program step.

Run tests step

Add a directory for RobotFramework tests to your repo and place all your tests there.

Add the final step that runs the tests.